

Student Attendance Management System

Project Overview: Modules & Technology Stack

1. Introduction

A Student Attendance Management System (SAMS) is a web-based application that records, tracks and reports student attendance digitally. It replaces paper registers with a central system where faculty mark attendance in a few clicks, percentages are calculated automatically, and students, parents and management can view accurate records and receive alerts when attendance runs low.

2. Objectives

- Replace manual attendance registers with a fast, accurate digital system.
- Calculate attendance percentages automatically, subject-wise and overall.
- Alert students and parents early when attendance falls below the required limit.
- Manage leave and on-duty requests with a proper approval flow.
- Give management ready-made reports, including defaulter lists.

3. User Roles

Role	Main Responsibilities
Admin	Manages users, departments, courses, classes, timetables, attendance rules and reports.
Faculty / Teacher	Marks attendance for assigned classes, approves leave, views class reports.
Student	Views own attendance, percentage and timetable; applies for leave.
Parent (optional)	Tracks the ward's attendance and receives shortage alerts.

4. System Modules

The system is divided into independent modules that share one database.

Module	Description	Key Features
1. Login & Authentication	Secure access for all user types.	Role-based login, password reset, session timeout, encrypted passwords.
2. Admin Dashboard	Central control panel.	Overview of attendance statistics, settings for minimum percentage and academic year.
3. Student Management	Maintains student records.	Registration, profile, roll number, class/section allocation, bulk import.

Module	Description	Key Features
4. Faculty Management	Maintains teacher records.	Faculty profile, subject and class allocation, workload.
5. Course, Class & Subject Management	Defines the academic structure.	Departments, courses, semesters, sections and subjects.
6. Timetable Management	Schedules classes and periods.	Class and faculty timetables, period slots, substitute faculty.
7. Attendance Marking	Records daily attendance.	Period-wise or day-wise marking, present/absent/late/on-duty, bulk mark, edit within allowed time.
8. Leave & On-Duty Management	Handles absence requests.	Online leave application, faculty approval, attendance auto-adjusted on approval.
9. Alerts & Shortage Tracking	Warns about low attendance.	Percentage calculation, shortage list, exam eligibility check, email/SMS alerts.
10. Reports & Analytics	Supports faculty and management.	Daily, monthly and subject-wise reports, defaulter list, charts, export to PDF/Excel.
11. Student / Parent Portal	Self-service view of records.	Attendance calendar, subject-wise percentage, leave status, timetable.

5. Attendance Workflow

1. Faculty logs in	2. Selects class & period	3. Marks attendance	4. Percentage updates	5. Alerts & reports
---------------------------	--------------------------------------	----------------------------	------------------------------	--------------------------------

6. System Architecture

The system follows a three-tier architecture:

Frontend Next.js (React) web app	Backend FastAPI: auth, business logic, database access and REST APIs	Database MySQL or MongoDB (backend-managed)
--	--	---

The Next.js frontend calls the FastAPI backend over REST APIs. The backend alone owns authentication, authorisation and all database access: it validates the JWT, applies role checks, reads and writes MySQL or MongoDB, recalculates percentages, triggers alerts, and returns JSON. The frontend holds no direct database or auth logic of its own.

7. Main Database Tables

users, roles, students, faculty, departments, courses, classes, subjects, timetable, attendance, leave_requests, alerts. All tables are created, migrated and accessed only through the FastAPI backend - the frontend never connects to the database directly.

8. Technology Stack

A commonly used stack for this project is shown below. Alternatives are listed so the stack can be matched to what your course or team prefers.

Layer	Technology (Primary)	Alternatives
Frontend	Next.js (React) with TypeScript, Tailwind CSS	Angular, Vue.js / Nuxt
Backend	Python FastAPI (async REST APIs); owns auth, business logic and all DB access	Django REST Framework, Node.js (Express/NestJS)
Database	MySQL (relational) or MongoDB (document) - choose one	PostgreSQL, SQL Server
ORM / Data Access	SQLAlchemy + Alembic (MySQL) or Motor / Beanie (MongoDB)	Prisma, Tortoise ORM
API Style	RESTful APIs with JSON, auto-generated docs (OpenAPI/Swagger)	GraphQL
Authentication	JWT issued and verified entirely in FastAPI; password hashing (passlib/bcrypt)	OAuth 2.0, session-based auth
Attendance Capture (optional)	QR code scanning (qrcode.js on frontend, verified by backend)	Biometric device, RFID, face recognition with Python + OpenCV
Notifications	FastAPI background tasks + SMTP (email alerts)	SMS gateway (Twilio), WhatsApp API
Reporting	Backend-generated exports: openpyxl (Excel), ReportLab (PDF); Chart.js on the frontend	JasperReports
Tools	VS Code, Git & GitHub, Postman, MySQL Workbench / MongoDB Compass	Eclipse, GitLab, XAMPP
Deployment	Uvicorn/Gunicorn for FastAPI + Vercel or Docker for Next.js; cloud hosting (AWS/Azure)	Heroku, Render, Railway

9. Benefits

- Saves faculty time and removes paper registers.
- Accurate, tamper-resistant attendance records with an audit trail.
- Early warning to students and parents before attendance becomes a problem.
- Instant reports for exam eligibility and management review.

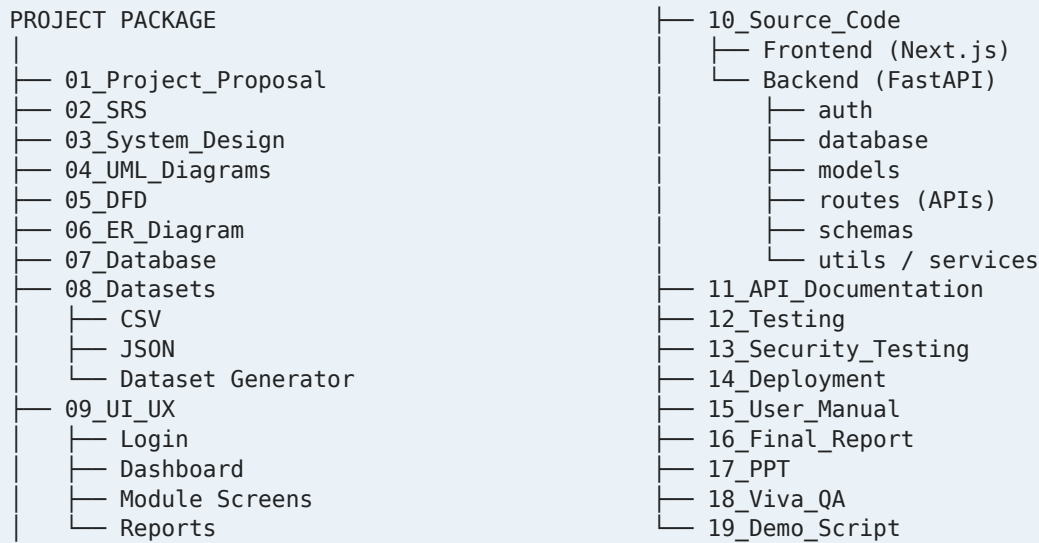
10. Future Enhancements

- Mobile app with QR or GPS-based attendance.
- Face-recognition or biometric attendance.
- AI-based prediction of students at risk of low attendance.

- WhatsApp and SMS alerts to parents.
- Integration with the College Management System and Online Examination System.

11. Project Package Structure

All project deliverables are organised into one package of 19 folders, from planning documents to source code, testing and final presentation.



Folder	Contents
01_Project_Proposal	Problem statement, objectives, scope and feasibility.
02_SRS	Functional and non-functional requirements (SRS).
03_System_Design	Architecture, module design and technology choices.
04_UML_Diagrams	Use case, class, sequence and activity diagrams.
05_DFD	Data flow diagrams (Level 0, 1 and 2).
06_ER_Diagram	Entity-relationship diagram of the database.
07_Database	Schema, SQL scripts and sample data.
08_Datasets	Sample students and attendance in CSV and JSON, plus a dataset generator.
09_UI_UX	Screens for Login, Dashboard, Modules and Reports.
10_Source_Code	Two top-level folders: Frontend (Next.js UI) and Backend (FastAPI). Everything else lives inside Backend: auth (login, JWT, password hashing), database (connection, models/ORM), models (data schemas), routes (REST API endpoints), schemas (request/response validation) and utils/services (business logic, alerts, reports).
11_API_Documentation	FastAPI auto-generated OpenAPI/Swagger docs for all REST endpoints.
12_Testing	Unit, integration and user acceptance test cases and results.
13_Security_Testing	Login, role-access, SQL injection, XSS and attendance-tampering tests.
14_Deployment	Server setup, configuration and deployment steps.
15_User_Manual	Step-by-step usage guide for admin, faculty, student and parent.
16_Final_Report	Complete project report.
17_PPT	Presentation slides.
18_Viva_QA	Expected viva questions with answers.
19_Demo_Script	Step-by-step flow for the live project demo.